



OUTLIERS

Verification

Layer 8 of 8 · Is it true, and is it working.
2 different questions.

FOR MAC

EST. MMXXVI · OUTLIERS ACCELERATOR

An automated system fails quietly by default. It carries on running, reporting success, and producing nothing, and there is nobody sitting next to you to notice.

Your system is fast, safe and ordered by a belief about what matters. Nothing has tested that belief. And nothing tells you whether the whole system is still doing its job, because a system with nothing to report and a system that has broken look identical from the outside.

2 checks. One asks whether what you are doing actually works, counted from your own records. The other asks whether your system is doing what you told it to, and says so loudly when it is not.

A diagram illustrating a logical relationship. At the top, there are two light beige rectangular boxes. The left box contains the text "WHAT YOU SAID" in black, with "should be true" in a smaller, lighter font below it. The right box contains the text "WHAT IS" in black, with "actually true" in a smaller, reddish-brown font below it. A dark red bracket connects the bottom of these two boxes, with a downward-pointing arrow leading to a dark red rectangular box at the bottom. This bottom box contains the text "THEY DISAGREE TELL ME" in white, all-caps font.

```
graph TD; A[WHAT YOU SAID  
should be true] --- B[WHAT IS  
actually true]; A --- B; B --> C[THEY DISAGREE TELL ME];
```

WHAT YOU SAID
should be true

WHAT IS
actually true

THEY DISAGREE TELL ME

2 QUESTIONS YOUR SYSTEM COULD NOT ANSWER UNTIL NOW.

How it works

There are 2 checks here. One looks at whether your work is producing results, and one looks at whether the software is still doing its job.

WHY YOU CANNOT USE ANYONE ELSE'S NUMBERS

A published figure about what works in sales has usually come from a company with something to sell, counting its own successes. That does not make it false, but it does mean it is measuring their customers, their market and their good days. It cannot tell you what works for you, and the only figures that can are your own.

COUNT ALL 4 BOXES, NOT JUST THE GOOD ONE

Say you believe replying quickly gets more calls booked. The tempting move is to count the people you replied to quickly who then booked a call. That number will look impressive and it means nothing on its own.

You need all 4: you did it and it worked, you did it and it did not, you did not do it and it worked anyway, and you did not do it and it did not work. Counting only the successes is how people end up with a superstition they are certain about.

Even with all 4, this will not prove that what you did caused the result. You may simply have replied fastest to the people who were already keen. What it does do is stop you being certain of something your own numbers do not support.

LOOKING BACK AT WHAT CAME BEFORE A RESULT

When something good happens, your system can now look backwards through your history and tell you what came before it. Sometimes the honest answer is that nothing you did was involved, which is worth knowing and slightly annoying.

THE SECOND CHECK: WHAT SHOULD BE TRUE

You write down, in plain English, a short list of statements that should be true of your system after it runs. Today's list should never be empty. Everyone on your do-not-contact list should still be on it. It should not have gone several runs without producing anything.

Then something checks that list against reality every time your system runs, and tells you when they disagree.

IT REPORTS. IT NEVER REPAIRS

The check has no ability to fix anything, deliberately. Something that quietly corrects what it finds is one more program changing your records with nobody watching, and you would never know what it had done. It tells you. You decide.

Before you start on a Mac

Everything in this guide is typed into **Terminal**, the Mac's window for typing commands instead of clicking. Press Command and Space together, type Terminal, and press Return.

This layer needs Python and git, which you set up for Layer 1 (its page "Before you start on a Mac"), and the layers before this one. It installs no add-ons, so it works with either Python a Mac may have.

HOW TO TYPE THE LINES

Where this guide shows several lines in a black box, type them one at a time: type the first, press Return, wait for it to finish and give you a fresh line, then type the next. The installer asks you questions. Each offers an answer in [square brackets]; press Return to take it, or type your own and press Return.

WHERE YOUR CRM IS

Layer 1 made your CRM at ~/CRM: a folder called CRM in your home folder (the folder with your name and the house picture in Finder's sidebar). If you chose another folder in Layer 1, put your own folder wherever a line in this guide says ~/CRM. The installer finds your CRM by itself either way.

Apple Silicon or Intel (the 2 kinds of chip a Mac can have; the Apple menu, then About This Mac, shows yours): the steps are the same on both, and both were tested.

Tried only on test Macs. Every step in this guide and in the instructions file of the layer's download was tried on test Macs (Macs GitHub rents out by the minute to run a script, not a person's own Mac, and not yours), never on a real Mac, except 1: the box asking to install the developer tools, which cannot appear on a test Mac. The last pages list every line and what happened to it.

Build it

3 lines, typed into Terminal. The last one asks you a few questions so what you get is yours, then does the work.

```
git clone https://github.com/OUTLIERS-ai/outliers-crm-08-verification-mac
cd outliers-crm-08-verification-mac
python3 install.py
```

This layer needs Layers 1 to 7. It checks the whole system, so it goes on last.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
What should always be true about your system?	In your own words. There should always be someone on today's list. Everyone I have put on my do-not-contact list should still be on it. 3 or 4 is plenty to start.
What would tell you it had stopped working?	Usually something producing nothing for a while, or a number going to 0 and staying there.

WHAT YOU END UP WITH

- A count of all 4 outcomes for anything you want to test, from your own records.
- A way to trace a result back through your history to what preceded it.
- A plain-English list of what should be true, which you own and can edit.
- A check that compares it to reality and tells you when they differ.

Now use it

Break it on purpose. It is the only way to know the alarm works.

The tools this layer installed live in a folder called **_engine** inside your CRM. In Terminal, go there first, then run the watchdog (if your CRM is not at ~/CRM, put your own folder in the cd line). Most of it should pass.

```
cd ~/CRM/_engine
python3 watchdog.py
```

Now break something deliberately. Empty your list, or take someone off your do-not-contact list who should be there. Run it again.

```
python3 watchdog.py
```

It catches you and says exactly what is wrong.

That is worth doing once, because from here on you are trusting this check to tell you when something has gone wrong, and an alarm nobody has ever heard is not an alarm.

IF YOU ALREADY HAD SOMETHING LIKE THIS

Your test: how would you currently find out that your system had stopped working? If the honest answer is that you would notice eventually, that is not a check, and it is the failure that costs the most because it costs you time you never knew you were losing.

WHAT THIS LAYER LEAVES UNSOLVED

Nothing new to build. But what you learn here changes the rules you wrote in Layer 2 and the sources you connected in Layer 4. Something you find out actually predicts a result earns a place in your system. A rule that keeps misfiring gets rewritten.

So you go back down the layers, change something near the bottom, and everything above it improves. That is not a sign the build is unfinished. A system that stops changing has stopped learning about the business it serves.

NEXT: LAYER 2 • THE RULES

Back to the bottom with what you have learned. This is the last guide, but it is not the end of the build; it is the point where you start improving the system from what it tells you, rather than waiting for the next instalment.

WORDS USED IN THIS GUIDE

All 4 boxes	Counting every combination of what you did and what happened, not just the successes.
Trace-back	Looking at what came before a result, in order.
Expectation	Something you have written down that should be true after your system runs.
Check	The program that compares your expectations to reality and reports the difference.

What was run on a Mac

A **test Mac** is one of the Macs GitHub rents out by the minute to run a script.

Each line below is one this guide or the instructions file of the layer's download (README.md) gives you to type.

The run was on 2026-09-26. A script (a list of commands each test Mac carries out by itself) downloaded a fresh copy of the download from GitHub on each of 7 test Macs and ran each line below exactly as printed, in this order. After a `cd` line (`cd` moves Terminal into a folder), the lines that follow ran in that folder, as they do for you. The script ran them; no person typed them.

7 test runs: 3 on Macs set up from scratch (macOS 15.7.9 on Apple Silicon, macOS 15.7.9 on Intel, macOS 26.6.2 on Apple Silicon), and 4 on 2 Macs that also have Homebrew, an add-on installer many Mac owners use, each run once with Homebrew installed before Python and once after (macOS 15.7.9 on Apple Silicon and macOS 15.7.9 on Intel). Before the first line, the script put these on every Mac: Python from python.org; Layers 1 to 7 of the CRM, installed from their Mac downloads. GitHub's Macs already have Apple's developer tools, which include git.

EACH LINE, THEN WHAT HAPPENED TO IT

```
git clone https://github.com/OUTLIERS-ai/outliers-crm-08-verification-mac
```

worked on all 7 test Macs

```
cd outliers-crm-08-verification-mac
```

worked on all 7 test Macs

```
python3 install.py
```

worked on all 7 test Macs

```
cd ~/CRM/_engine
```

worked on all 7 test Macs

```
python3 watchdog.py
```

worked on all 7 test Macs

```
cd ~/CRM
```

worked on all 7 test Macs

```
python3 _engine/fourbox.py reply_in meeting_booked --within 30
```

worked on all 7 test Macs

```
python3 _engine/watchdog.py
```

worked on all 7 test Macs

```
cd ~/outliers-crm-08-verification-mac
```

worked on all 7 test Macs

EACH LINE, CONTINUED

```
python3 tests/test_fourbox.py
```

worked on all 7 test Macs

```
python3 tests/test_watchdog.py
```

worked on all 7 test Macs

The script's record of the run is public, for anyone who wants to check it; you do not need to open it:

<https://github.com/OUTLIERS-ai/outliers-mac-test/actions/runs/36210576465>

NOT RUN ON THE TEST MACS

The box asking to install the developer tools cannot appear on a test Mac, because GitHub's Macs already have them.